

Term Rewriting with Shared Memory

Mikhail Ushakov

Radboud University Nijmegen

Supervisor : Dr C.L.M. Kop

June 2, 2026

Why Should We Care?

web servers

databases

concurrent
graph algorithms

parallel sorting
operations

operating systems

Why Should We Care?

web servers

databases

concurrent
graph algorithms

Shared data

parallel sorting
operations

operating systems

Why Should We Care?

Concurrent algorithms and data structures are **fundamental** in modern world

Why Should We Care?

Concurrent algorithms and data structures are **fundamental** in modern world

Concurrency gives **performance benefits**, but introduces **extra complexity**

Why Should We Care?

Concurrent algorithms and data structures are **fundamental** in modern world

Concurrency gives **performance benefits**, but introduces **extra complexity**

Concurrent software requires **careful design** and **formal reasoning**

What Can Help Us?

We are looking for a framework that would allow us to do:

What Can Help Us?

We are looking for a framework that would allow us to do:

- ✓ Program **termination analysis**

What Can Help Us?

We are looking for a framework that would allow us to do:

- ✓ Program **termination analysis**
- ✓ Program **equivalence checking**

What Can Help Us?

We are looking for a framework that would allow us to do:

- ✓ Program **termination analysis**
- ✓ Program **equivalence checking**
- ✓ **Formal verification** of programs and models

What Can Help Us?

Term Rewriting Systems (TRS)

What Can Help Us?

Logically Constrained Term Rewriting Systems (LCTRS)

The End?

The End?

The End?

This is Not The End!

This is Not The End!

✗ Traditional TRS **lacks** direct support for shared mutable memory

This is Not The End!

- ✗ Traditional TRS **lacks** direct support for shared mutable memory
- ✗ Existing approaches **do not** combine parallel rewriting with explicit memory effects

This is Not The End!

- ✗ Traditional TRS **lacks** direct support for shared mutable memory
- ✗ Existing approaches **do not** combine parallel rewriting with explicit memory effects
- ✗ This **limits** rewriting-based modelling of concurrent data structures

Goal of the Research Project

Main Research Question

Can we extend term rewriting with explicit shared memory so that it is **formal**, **implementable**, and **useful for modelling concurrent algorithms**?

- ☐ Introduce **Memory Term Rewriting Systems (MemTRSs)** as an extension of LCTRSs.
- ☐ Add explicit shared-memory operations
- ☐ Evaluate the expressiveness and practicality of this model through data structures, algorithms, and experiments.

LCTRS Recap: Rewriting Rules

$$r_1 : \mathbf{max}(a, \mathbf{cons}(b, l)) \rightarrow \mathbf{max}(b, l) [a < b]$$

$$r_2 : \mathbf{max}(a, \mathbf{cons}(b, l)) \rightarrow \mathbf{max}(a, l) [a \geq b]$$

$$r_3 : \mathbf{max}(a, \mathbf{nil}) \rightarrow a$$

LCTRS Recap: Reduction Example

max(-1, **cons**(2, **cons**(3, **cons**(0, nil))))

LCTRS Recap: Reduction Example

max(-1, **cons**(2, **cons**(3, **cons**(0, nil))))
 $\xrightarrow{r1}$ **max**(2, **cons**(3, **cons**(0, nil)))

LCTRS Recap: Reduction Example

max(-1, **cons**(2, **cons**(3, **cons**(0, nil))))

$\xrightarrow{r1}$ **max**(2, **cons**(3, **cons**(0, nil))) (because $-1 < 2$)

$\xrightarrow{r1}$ **max**(3, **cons**(0, nil))

LCTRS Recap: Reduction Example

max(-1, **cons**(2, **cons**(3, **cons**(0, nil))))

$\xrightarrow{r1}$ **max**(2, **cons**(3, **cons**(0, nil))) (because $-1 < 2$)

$\xrightarrow{r1}$ **max**(3, **cons**(0, nil)) (because $2 < 3$)

$\xrightarrow{r2}$ **max**(3, nil)

LCTRS Recap: Reduction Example

max(-1, **cons**(2, **cons**(3, **cons**(0, nil))))

$\xrightarrow{r1}$ **max**(2, **cons**(3, **cons**(0, nil))) (because $-1 < 2$)

$\xrightarrow{r1}$ **max**(3, **cons**(0, nil)) (because $2 < 3$)

$\xrightarrow{r2}$ **max**(3, nil) (because $3 \geq 0$)

$\xrightarrow{r3}$ 3

Where is Shared State? MemTRS Core Idea

term

Where is Shared State? MemTRS Core Idea

$\langle \textit{term}, \textit{mem} \rangle$

Where is Shared State? MemTRS Core Idea

$\langle term, mem \rangle$

$mem : \mathbb{Z} \rightarrow \mathbb{Z}$

Kernel Memory Operations

GET : $[int] \Rightarrow int$

Reads from memory.

If x is valid: $\langle \mathbf{GET}(x), mem \rangle \rightarrow \langle mem(x), mem \rangle$

SET : $[int \times int] \Rightarrow bool$

Writes to memory.

If x is valid: $\langle \mathbf{SET}(x, v), mem \rangle \rightarrow \langle true, mem[x \mapsto v] \rangle$

CAS : $[int \times int \times int] \Rightarrow bool$

Atomic compare-and-swap.

If $mem(x) = e$: $\langle \mathbf{CAS}(x, e, v), mem \rangle \rightarrow \langle true, mem[x \mapsto v] \rangle$

Parallel Reduction

Memory

$mem(0) = 0$

$mem(1) = 0$

$s(\mathbf{SET}(0, 3), \mathbf{SET}(1, 4))$

Parallel Reduction

Memory

$mem(0) = 3$

$mem(1) = 4$

$s(\text{SET}(0, 3), \text{SET}(1, 4))$

true

true

Parallel Reduction

Memory

$mem(0) = 3$

$mem(1) = 4$

$s(\text{ true }, \text{ true })$

Nondeterminism from Parallel Writes

Memory

$mem(0) = 0$

$s(\mathbf{SET}(0, 3), \mathbf{SET}(0, 4))$

Nondeterminism from Parallel Writes

Memory

$mem(0) = 3$

$s(\text{SET}(0, 3), \text{SET}(0, 4))$



true

Nondeterminism from Parallel Writes

Memory

$mem(0) = 3$

$s(\text{true}, \mathbf{SET}(0, 4))$

Nondeterminism from Parallel Writes

Memory

$mem(0) = 4$

`s( true , SET(0,4))`



true

Nondeterminism from Parallel Writes

Memory

$mem(0) = 4$

$s(\text{true}, \text{true})$

Nondeterminism from Parallel Writes

Memory

$mem(0) = 0$

$s(\mathbf{SET}(0, 3), \mathbf{SET}(0, 4))$

Nondeterminism from Parallel Writes

Memory

$mem(0) = 4$

$s(\mathbf{SET}(0, 3), \mathbf{SET}(0, 4))$



`true`

Nondeterminism from Parallel Writes

Memory

$mem(0) = 4$

$s(\mathbf{SET}(0, 3), \text{true})$

Nondeterminism from Parallel Writes

Memory

$mem(0) = 3$

$s(\text{SET}(0, 3), \text{true})$



true

Nondeterminism from Parallel Writes

Memory

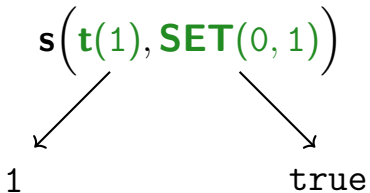
$mem(0) = 3$

$s(\text{true}, \text{true})$

Innermost Reduction Strategy

 $s(x, y) \rightarrow x$ $t(x) \rightarrow x$ **Memory** $mem(0) = 0$ $s(t(1), \mathbf{SET}(0, 1))$

Innermost Reduction Strategy

 $s(x, y) \rightarrow x$ $t(x) \rightarrow x$ **Memory** $mem(0) = 0$ 

Innermost Reduction Strategy

 $s(x, y) \rightarrow x$ $t(x) \rightarrow x$ **Memory** $mem(0) = 1$ $s(1, \text{true})$

Innermost Reduction Strategy

 $s(x, y) \rightarrow x$ $t(x) \rightarrow x$ **Memory** $mem(0) = 1$ $s(1, true)$ 

1

Innermost Reduction Strategy

 $s(x, y) \rightarrow x$ $t(x) \rightarrow x$ **Memory** $mem(0) = 1$

1

Outermost Reduction Strategy

Rules

 $s(x, y) \rightarrow x$ $t(x) \rightarrow x$

Memory

 $mem(0) = 0$ $s(t(1), \mathbf{SET}(0, 1))$

Outermost Reduction Strategy

Rules

 $s(x, y) \rightarrow x$ $t(x) \rightarrow x$

Memory

 $mem(0) = 0$ $s(t(1), \mathbf{SET}(0, 1))$

Outermost Reduction Strategy

Rules

 $s(x, y) \rightarrow x$ $t(x) \rightarrow x$

Memory

 $mem(0) = 0$ $s(t(1), \mathbf{SET}(0, 1))$  $t(1)$

Outermost Reduction Strategy

Rules

 $s(x, y) \rightarrow x$ $t(x) \rightarrow x$

Memory

 $mem(0) = 0$ $t(1)$

Outermost Reduction Strategy

Rules

 $s(x, y) \rightarrow x$ $t(x) \rightarrow x$

Memory

 $mem(0) = 0$ $t(1)$

Outermost Reduction Strategy

Rules

 $s(x, y) \rightarrow x$ $t(x) \rightarrow x$

Memory

 $mem(0) = 0$ $t(1)$ 

1

Outermost Reduction Strategy

Rules

 $s(x, y) \rightarrow x$ $t(x) \rightarrow x$

Memory

 $mem(0) = 0$

1

Reflection: Design Evolution of Memory Operations

- Initial idea: one-address **READ, WRITE**

Reflection: Design Evolution of Memory Operations

- Initial idea: one-address **READ, WRITE**
- Too limited for realistic data structures

Reflection: Design Evolution of Memory Operations

- Initial idea: one-address **READ**, **WRITE**
- Too limited for realistic data structures
- Next idea: make **GET**, **SET**, and **CAS** logical/theory symbols

Reflection: Design Evolution of Memory Operations

- Initial idea: one-address **READ**, **WRITE**
- Too limited for realistic data structures
- Next idea: make **GET**, **SET**, and **CAS** logical/theory symbols
- Problem: they could appear inside rule constraints

Reflection: Design Evolution of Memory Operations

$$\mathbf{f}(x) \rightarrow \mathbf{g}(x) [\mathbf{GET}(0) = 1]$$

Reflection: Design Evolution of Memory Operations

$$\mathbf{f}(x) \rightarrow \mathbf{g}(x) [\mathbf{SET}(0, 1) = \text{true} \wedge x > 0]$$

Reflection: Design Evolution of Memory Operations

~~$f(x) \rightarrow g(x) [\text{SET}(0, 1) \equiv \text{true} \wedge x > 0]$~~

Reflection: Design Evolution of Memory Operations

- Initial idea: one-address **READ**, **WRITE**
- Too limited for realistic data structures
- Next idea: make **GET**, **SET**, and **CAS** logical/theory symbols
- Problem: they could appear inside rule constraints
- Final lesson: memory operations should be explicit memory-reduction steps

MemTRS in Cora: Prototype REPL

```
Welcome to MemTRS REPL v0.1
```

```
memtrs>
```

MemTRS in Cora: Prototype REPL

```
Welcome to MemTRS REPL v0.1
```

```
memtrs> include memtrs/stdlib/mem_ds/array.lctrs
```

```
OK
```

```
memtrs>
```

MemTRS in Cora: Prototype REPL

```
Welcome to MemTRS REPL v0.1
```

```
memtrs> include memtrs/stdlib/mem_ds/array.lctrs
```

```
OK
```

```
memtrs> include memtrs/stdlib/algorithms/quicksort.lctrs
```

```
OK
```

```
memtrs>
```


MemTRS in Cora: Prototype REPL

Welcome to MemTRS REPL v0.1

```
memtrs> include memtrs/stdlib/mem_ds/array.lctrs
```

OK

```
memtrs> include memtrs/stdlib/algorithms/quicksort.lctrs
```

OK

```
memtrs> reduce listToArray(cons(3, cons(1, cons(5, cons(2, cons(4, nil))))))
```

Reduced to: 2

```
memtrs>
```

MemTRS in Cora: Prototype REPL

```
Welcome to MemTRS REPL v0.1
```

```
memtrs> include memtrs/stdlib/mem_ds/array.lctrs
```

```
OK
```

```
memtrs> include memtrs/stdlib/algorithms/quicksort.lctrs
```

```
OK
```

```
memtrs> reduce listToArray(cons(3, cons(1, cons(5, cons(2, cons(4, nil))))))
```

```
Reduced to: 2
```

```
memtrs> print_mem
```

```
Memory:
```

```
[8, 0, 5, 3, 1, 5, 2, 4, ...]
```

```
memtrs>
```

MemTRS in Cora: Prototype REPL

```
Welcome to MemTRS REPL v0.1
```

```
memtrs> include memtrs/stdlib/mem_ds/array.lctr
```

```
OK
```

```
memtrs> include memtrs/stdlib/algorithms/quicksort.lctr
```

```
OK
```

```
memtrs> reduce listToArray(cons(3, cons(1, cons(5, cons(2, cons(4, nil))))))
```

```
Reduced to: 2
```

```
memtrs> print_mem
```

```
Memory:
```

```
[8, 0, 5, 3, 1, 5, 2, 4, ...]
```

```
memtrs> reduce qsortArray(2, 0, 4)
```

```
Reduced to: true
```

```
memtrs>
```

MemTRS in Cora: Prototype REPL

```
Welcome to MemTRS REPL v0.1
```

```
memtrs> include memtrs/stdlib/mem_ds/array.lctr
```

```
OK
```

```
memtrs> include memtrs/stdlib/algorithms/quicksort.lctr
```

```
OK
```

```
memtrs> reduce listToArray(cons(3, cons(1, cons(5, cons(2, cons(4, nil))))))
```

```
Reduced to: 2
```

```
memtrs> print_mem
```

```
Memory:
```

```
[8, 0, 5, 3, 1, 5, 2, 4, ...]
```

```
memtrs> reduce qsortArray(2, 0, 4)
```

```
Reduced to: true
```

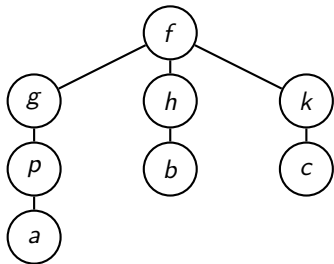
```
memtrs> print_mem
```

```
Memory:
```

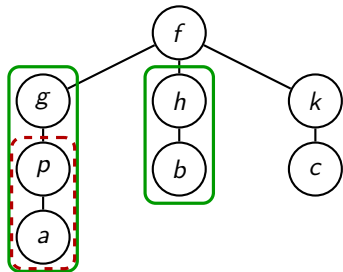
```
[8, 0, 5, 1, 2, 3, 4, 5, ...]
```

```
memtrs>
```

Parallel Rewriting in Cora


$$f(g(p(a)), h(b), k(c))$$

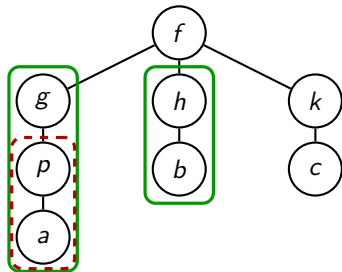
Parallel Rewriting in Cora



Find possible rewrites

$f(g(p(a)), h(b), k(c))$

Parallel Rewriting in Cora

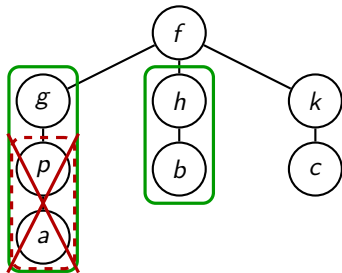

$$f(g(p(a)), h(b), k(c))$$

Find possible rewrites



Try them in parallel
using virtual threads

Parallel Rewriting in Cora

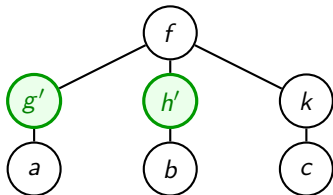

$$f(g(p(a)), h(b), k(c))$$

Find possible rewrites

Try them in parallel
using virtual threads

Remove conflicts
with a prefix-free trie

Parallel Rewriting in Cora



$$f(g(p(a)), h(b), k(c)) \Rightarrow f(g'(a), h'(b), k(c))$$

Find possible rewrites

Try them in parallel
using virtual threads

Remove conflicts
with a prefix-free trie

Build the next term

Reflection: Implementation Trade-offs in Cora

- Early memory implementations were either **unsafe** or **inefficient**

Reflection: Implementation Trade-offs in Cora

- Early memory implementations were either **unsafe** or **inefficient**
- ConcurrentMap solved thread safety, but added too much **overhead**

Reflection: Implementation Trade-offs in Cora

- Early memory implementations were either **unsafe** or **inefficient**
- ConcurrentMap solved thread safety, but added too much **overhead**
- AtomicIntegerArray was less flexible, but more **predictable** and **efficient**

Reflection: Implementation Trade-offs in Cora

- Early memory implementations were either **unsafe** or **inefficient**
- ConcurrentMap solved thread safety, but added too much **overhead**
- AtomicIntegerArray was less flexible, but more **predictable** and **efficient**
- Parallel reduction had to be corrected to rewrite only **disjoint** redexes

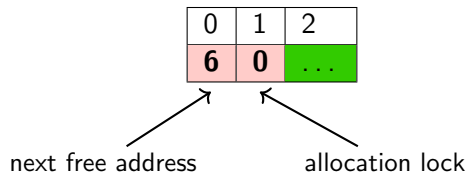
Reflection: Implementation Trade-offs in Cora

- Early memory implementations were either **unsafe** or **inefficient**
- ConcurrentMap solved thread safety, but added too much **overhead**
- AtomicIntegerArray was less flexible, but more **predictable** and **efficient**
- Parallel reduction had to be corrected to rewrite only **disjoint** redexes
- **Virtual threads** improved the practical behavior of parallel rewriting

Reflection: Implementation Trade-offs in Cora

- Early memory implementations were either **unsafe** or **inefficient**
- ConcurrentMap solved thread safety, but added too much **overhead**
- AtomicIntegerArray was less flexible, but more **predictable** and **efficient**
- Parallel reduction had to be corrected to rewrite only **disjoint** redexes
- **Virtual threads** improved the practical behavior of parallel rewriting
- The experiments reflect the **Cora-based implementation**, not only abstract MemTRS model

MemTRS: Global Memory Model



Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
2	0	...	...	...

`s(        alloc(1)        ,        alloc(1)        )`

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
2	0	...	...	...

$s(\text{alloc}(1), \text{alloc}(1))$

$\text{allocLock}(1, \text{CAS}(1, 0, 1))$

$\text{allocLock}(1, \text{CAS}(1, 0, 1))$

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
2	0	...	...	...

$s(\text{allocLock}(1, \text{CAS}(1, 0, 1)), \text{allocLock}(1, \text{CAS}(1, 0, 1)))$

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
2	0	...	...	...

`s(allocLock(1, CAS(1, 0, 1)), allocLock(1, CAS(1, 0, 1)))`

true

false

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
2	1	...	...	...

`s(allocLock(1, true ), allocLock(1, false ))`

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
2	1	...	...	...

`s( allocLock(1,true) , allocLock(1,false) )`



`allocRead(1, GET(0))`



`alloc(1)`

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
2	1	...	...	...

$s(\text{allocRead}(1, \mathbf{GET}(0)), \quad \text{alloc}(1) \quad)$

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
2	1	...	...	...

`s(allocRead(1, GET(0)),`



`alloc(1) )`



`allocLock(1, CAS(1, 0, 1))`

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
2	1	...	...	...

$s(\text{allocRead}(1, \text{2}), \text{allocLock}(1, \text{CAS}(1, 0, 1)))$

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
2	1	...	...	...

Diagram illustrating the execution of the `allocRead` and `allocLock` functions:

```
graph TD; A["s( allocRead(1, 2) , allocLock(1, CAS(1, 0, 1)))"] --> B["allocReserve(2, SET(0, 2 + 1))"]; A --> C["false"];
```

The diagram shows the function call `s( allocRead(1, 2) , allocLock(1, CAS(1, 0, 1)))`. Arrows indicate the return values of the sub-expressions:

- `allocRead(1, 2)` returns `allocReserve(2, SET(0, 2 + 1))`.
- `allocLock(1, CAS(1, 0, 1))` returns `false`.

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
2	1	...	...	...

`s(allocReserve(2, SET(0, 2 + 1)), allocLock(1, false ))`

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
2	1	...	...	...

`s(allocReserve(2, SET(0, 2 + 1)), allocLock(1, false) )`

↓

3

↓

`alloc(1)`

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
2	1	...	...	...

`s(allocReserve(2, SET(0, 3)), alloc(1))`

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
2	1	...	...	...

`s(allocReserve(2, SET(0, 3)),`

↙
`true`

`alloc(1)                    )`

↘
`allocLock(1, CAS(1, 0, 1))`

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
3	1	...	...	...

`s(allocReserve(2, true), allocLock(1, CAS(1, 0, 1)))`

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
3	1	...	...	...

`s( allocReserve(2, true) , allocLock(1, CAS(1, 0, 1)))`

`allocUnlock(2, SET(1, 0))`

`false`

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
3	1	...	...	...

`s(allocUnlock(2, SET(1,0)), allocLock(1, false ))`

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
3	1	...	...	...

`s(allocUnlock(2, SET(1,0)), allocLock(1,false) )`

`true`

`alloc(1)`

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
3	0	...	...	...

`s(allocUnlock(2, true ), alloc(1) )`

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
3	0	...	...	...

`s(allocUnlock(2, true),`

↙
2

`alloc(1)                    )`

↘

`allocLock(1, CAS(1, 0, 1))`

Memory Reduction: Parallel Allocation I

Memory

0	1	2	3	4
3	0	...	...	...

$s($ 2 $, \text{allocLock}(1, \text{CAS}(1, 0, 1)))$

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
3	0	...	...	...

$s(2, \text{allocLock}(1, \text{CAS}(1, 0, 1)))$

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
3	0	...	...	...

$s(2, \text{allocLock}(1, \text{CAS}(1, 0, 1)))$



true

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
3	1	...	...	...

`s(2, allocLock(1, true ))`

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
3	1	...	...	...

$s(2, \text{allocLock}(1, \text{true}))$



`allocRead(1, GET(0))`

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
3	1	...	...	...

$s(2, \text{allocRead}(1, \mathbf{GET}(0)))$

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
3	1	...	...	...

$s(2, \text{allocRead}(1, \text{GET}(0)))$



3

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
3	1	...	...	...

$s(2, \text{allocRead}(1, \text{3}))$

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
3	1	...	...	...

$s(2,$ **allocRead**(1, 3))



allocReserve(3, **SET**(0, 3 + 1))

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
3	1	...	...	...

$s(2, \text{allocReserve}(3, \text{SET}(0, 3 + 1)))$

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
3	1	...	...	...

$s(2, \text{allocReserve}(3, \text{SET}(0, 3 + 1)))$



4

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
3	1	...	...	...

$s(2, \text{allocReserve}(3, \text{SET}(0, 4)))$

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
3	1	...	...	...

$s(2, \text{allocReserve}(3, \text{SET}(0, 4)))$



true

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
4	1	...	...	...

$s(2, \text{allocReserve}(3, \text{true}))$

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
4	1	...	...	...

$s(2, \text{allocReserve}(3, \text{true}))$



$\text{allocUnlock}(3, \text{SET}(1, 0))$

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
4	1	...	...	...

$s(2, \text{allocUnlock}(3, \text{SET}(1, 0)))$

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
4	1	...	...	...

$s(2, \text{allocUnlock}(3, \text{SET}(1, 0)))$



true

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
4	0	...	...	...

`s(2, allocUnlock(3, true ))`

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
4	0	...	...	...

$s(2, \text{allocUnlock}(3, \text{true}))$



3

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
4	0	...	...	...

$s(2, \quad 3 \quad)$

Memory Reduction: Parallel Allocation II

Memory

0	1	2	3	4
4	0	...	...	...

$s(2, 3)$

Reflection: Memory Gives Expressiveness, but Adds Low-Level Detail

- Shared memory made arrays, matrices, queues, and graph algorithms **easier** to model

Reflection: Memory Gives Expressiveness, but Adds Low-Level Detail

- Shared memory made arrays, matrices, queues, and graph algorithms **easier** to model
- But the model became more **explicit** and **lower-level**

Reflection: Memory Gives Expressiveness, but Adds Low-Level Detail

- Shared memory made arrays, matrices, queues, and graph algorithms **easier** to model
- But the model became more **explicit** and **lower-level**
- I had to **reason** about addresses, allocation, evaluation order, and synchronization

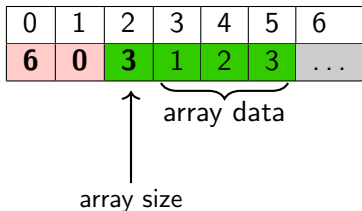
Reflection: Memory Gives Expressiveness, but Adds Low-Level Detail

- Shared memory made arrays, matrices, queues, and graph algorithms **easier** to model
- But the model became more **explicit** and **lower-level**
- I had to **reason** about addresses, allocation, evaluation order, and synchronization
- Some algorithms and procedures started to look closer to **imperative programs** encoded as rewrite rules

Reflection: Memory Gives Expressiveness, but Adds Low-Level Detail

- Shared memory made arrays, matrices, queues, and graph algorithms **easier** to model
- But the model became more **explicit** and **lower-level**
- I had to **reason** about addresses, allocation, evaluation order, and synchronization
- Some algorithms and procedures started to look closer to **imperative programs** encoded as rewrite rules
- This raises the question: can MemTRS keep the declarative strengths of rewriting?

Arrays in MemTRS



Arrays in MemTRS

[7, 2, 10, 4, 1, 9, 3, 6, 8, 5]

Arrays in MemTRS

0	1	2	3	4	5	6	7	8	9	10	11	12
13	0	10	7	2	10	4	1	9	3	6	8	5

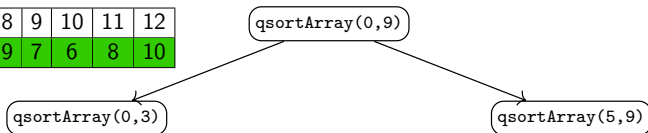
QuickSort with Memory

0	1	2	3	4	5	6	7	8	9	10	11	12
13	0	10	7	2	10	4	1	9	3	6	8	5

qsortArray(0,9)

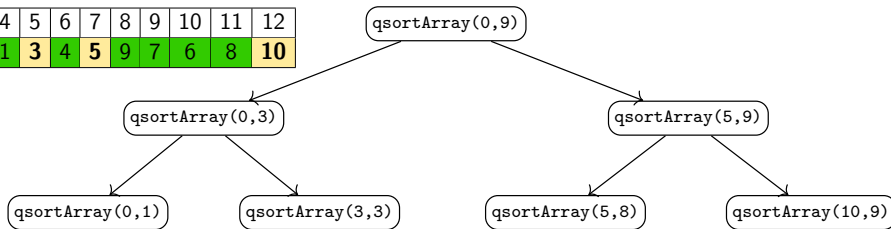
QuickSort with Memory

0	1	2	3	4	5	6	7	8	9	10	11	12
13	0	10	2	4	1	3	5	9	7	6	8	10



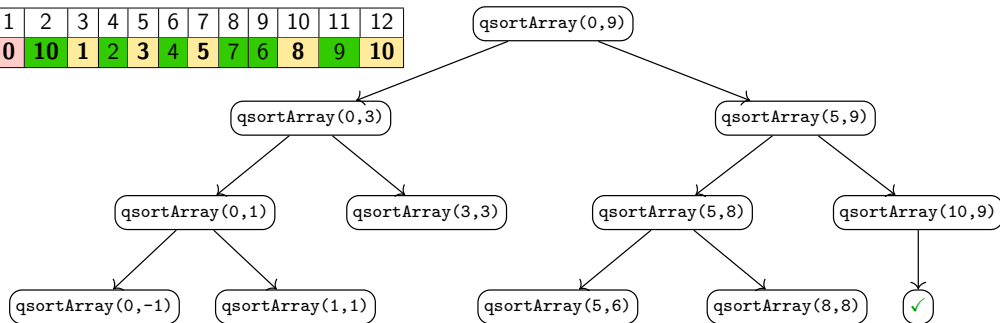
QuickSort with Memory

0	1	2	3	4	5	6	7	8	9	10	11	12
13	0	10	2	1	3	4	5	9	7	6	8	10



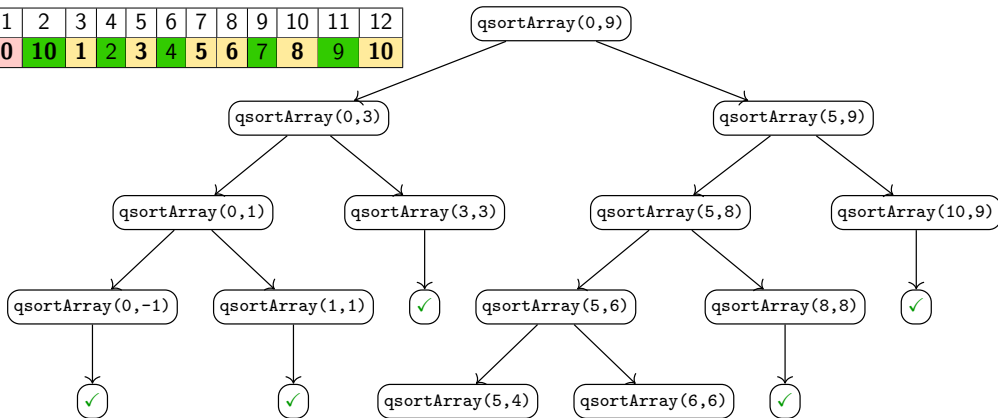
QuickSort with Memory

0	1	2	3	4	5	6	7	8	9	10	11	12
13	0	10	1	2	3	4	5	7	6	8	9	10



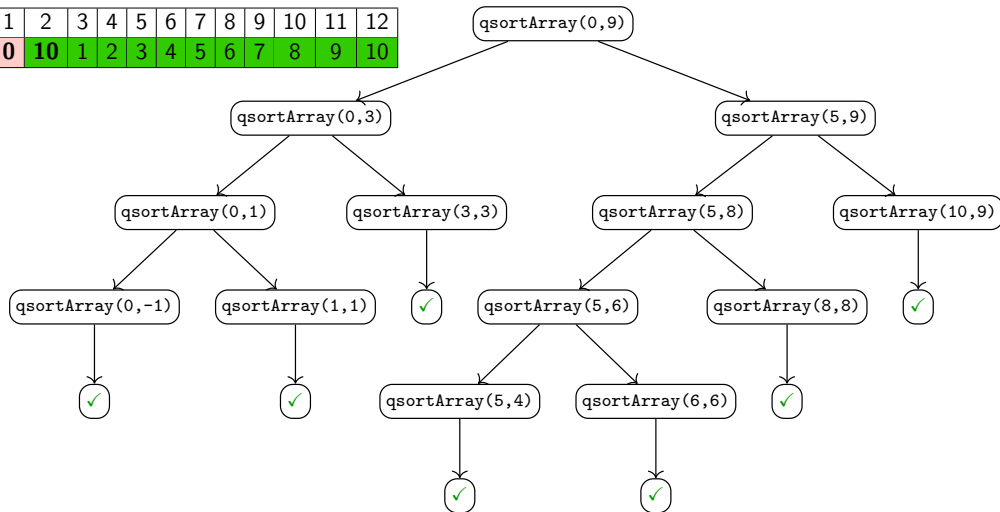
QuickSort with Memory

0	1	2	3	4	5	6	7	8	9	10	11	12
13	0	10	1	2	3	4	5	6	7	8	9	10

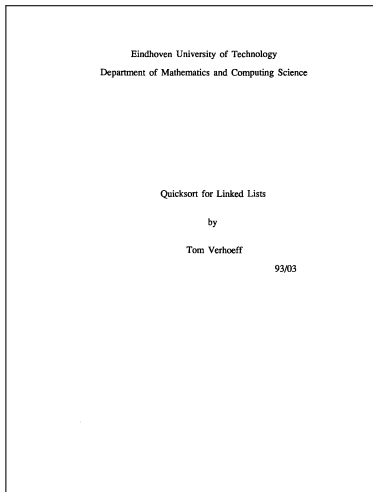


QuickSort with Memory

0	1	2	3	4	5	6	7	8	9	10	11	12
13	0	10	1	2	3	4	5	6	7	8	9	10



QuickSort without Memory: Comparison Baseline



QuickSort without Memory: Comparison Baseline

Input: `qsortList([7, 2, 10, 4, 1, 9, 3, 6, 8, 5], nil)`

QuickSort without Memory: Comparison Baseline

Input: `qsortList([7, 2, 10, 4, 1, 9, 3, 6, 8, 5], nil)`

First partition:

pivot: **7**

left: [5, 6, 3, 1, 4, 2]

right: [8, 9, 10]

QuickSort without Memory: Comparison Baseline

Input: `qsortList([7, 2, 10, 4, 1, 9, 3, 6, 8, 5], nil)`

First partition:

pivot: **7**

left: [5, 6, 3, 1, 4, 2]

right: [8, 9, 10]

Rewritten term: `qsortList([5, 6, 3, 1, 4, 2], 7 :: qsortList([8, 9, 10], nil))`

QuickSort without Memory: Comparison Baseline

Input: `qsortList([7, 2, 10, 4, 1, 9, 3, 6, 8, 5], nil)`

First partition:

pivot: **7**

left: [5, 6, 3, 1, 4, 2]

right: [8, 9, 10]

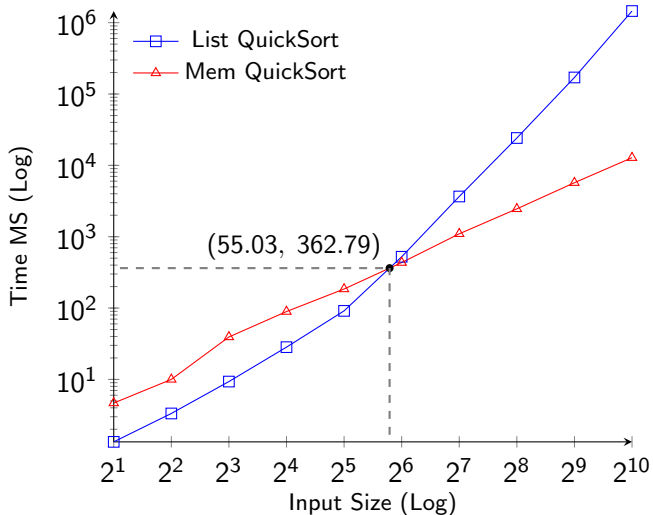
Rewritten term: `qsortList([5, 6, 3, 1, 4, 2], 7 :: qsortList([8, 9, 10], nil))`

Reduced to: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

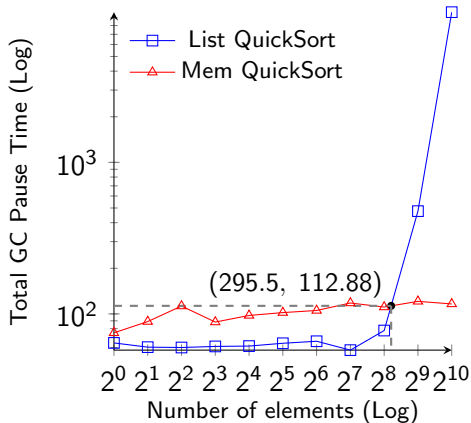
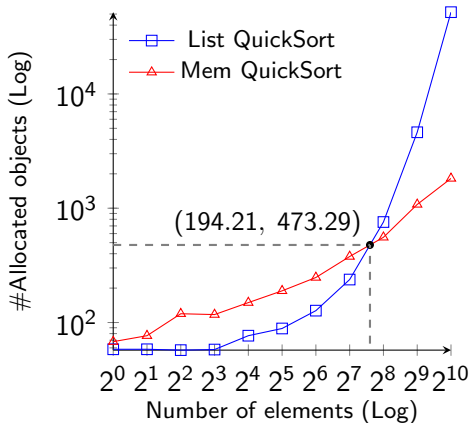
QuickSort Case Study: Experiment Setup

- Comparing two implementations of QuickSort (MemTRS vs LCTRS)
- Both implementations have average runtime complexity of $\mathcal{O}(n \log n)$
- Randomly generated integer arrays
- Input sizes: 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024
- Repeated 3 times each

QuickSort Case Study: Runtime Results



QuickSort Case Study: Profiling with JFR



QuickSort Case Study: CPU-Time Profiling

List QuickSort

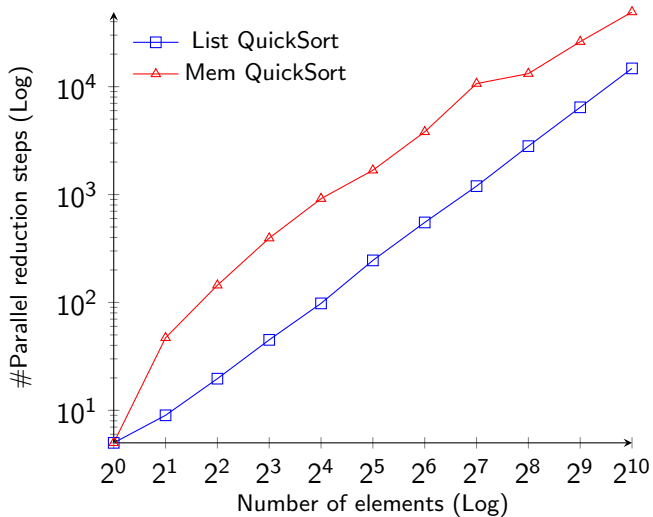
#	Thread	Samples	Percent
1	main	51,010	65.58%
2	<i>virtual</i>	13,992	17.99%
3–14	ForkJoinPool workers	12,716	14.89%
15–19	Other	68	0.09%

QuickSort Case Study: CPU-Time Profiling

Memory QuickSort

#	Thread	Samples	Percent
1	<i>virtual</i>	8,692	63.55%
2	main	2,394	17.50%
3–14	ForkJoinPool workers	2578	18.86%
15–16	Other	13	0.10%

QuickSort Case Study: Parallel Reduction Steps



QuickSort Case Study: Interpretation of Results

- List QuickSort is faster for **small** inputs

QuickSort Case Study: Interpretation of Results

- List QuickSort is faster for **small** inputs
- Memory QuickSort performs better for **larger** inputs

QuickSort Case Study: Interpretation of Results

- List QuickSort is faster for **small** inputs
- Memory QuickSort performs better for **larger** inputs
- Both versions show **similar** asymptotic growth in parallel reduction steps

QuickSort Case Study: Interpretation of Results

- List QuickSort is faster for **small** inputs
- Memory QuickSort performs better for **larger** inputs
- Both versions show **similar** asymptotic growth in parallel reduction steps
- Main difference comes from **engine-level overhead**

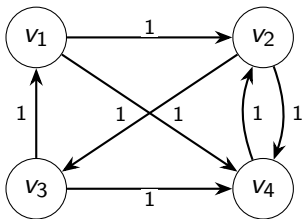
QuickSort Case Study: Interpretation of Results

- List QuickSort is faster for **small** inputs
- Memory QuickSort performs better for **larger** inputs
- Both versions show **similar** asymptotic growth in parallel reduction steps
- Main difference comes from **engine-level overhead**
- List QuickSort stores input inside **large terms**, causing expensive subterm processing

QuickSort Case Study: Interpretation of Results

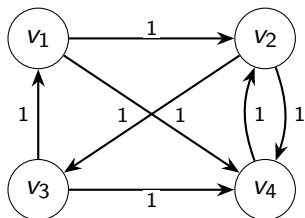
- List QuickSort is faster for **small** inputs
- Memory QuickSort performs better for **larger** inputs
- Both versions show **similar** asymptotic growth in parallel reduction steps
- Main difference comes from **engine-level overhead**
- List QuickSort stores input inside **large terms**, causing expensive subterm processing
- Memory QuickSort keeps array data in global memory, **reducing** allocation pressure

Matrices in MemTRS: Moving towards Graph Algorithms



	v_1	v_2	v_3	v_4
v_1	0	1	∞	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Matrices in MemTRS: Moving towards Graph Algorithms



0	1	2	3	4	5	6
27	0	4	7	12	17	22

matrix rows

7	8	9	10	11
4	0	1	∞	1
12	13	14	15	16
4	∞	0	1	1
17	18	19	20	21
4	1	∞	0	1
22	23	24	25	26
4	∞	1	∞	0

Parallel Floyd–Warshall

	W1		W2	
	v_1	v_2	v_3	v_4
v_1	0	1	∞	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0
	W3		W4	

Parallel Floyd–Warshall

	W1		W2	
	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0
	W3		W4	

Parallel Floyd–Warshall

	W1		W2	
	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0
	W3		W4	

Parallel Floyd–Warshall

	W1		W2	
	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	2	0
	W3		W4	

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	∞	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	∞	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	∞	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	∞	0

Sequential Floyd–Warshall: Comparison Baseline

single worker

	v_1	v_2	v_3	v_4
v_1	0	1	2	1
v_2	∞	0	1	1
v_3	1	∞	0	1
v_4	∞	1	2	0

Floyd–Warshall Case Study: Experiment Setup

- Comparing two Floyd–Warshall encodings: parallel vs sequential

Floyd–Warshall Case Study: Experiment Setup

- Comparing two Floyd–Warshall encodings: parallel vs sequential
- Both versions store the distance matrix in memory

Floyd–Warshall Case Study: Experiment Setup

- Comparing two Floyd–Warshall encodings: parallel vs sequential
- Both versions store the distance matrix in memory
- Sequential version uses one worker; parallel version uses 4 block workers

Floyd–Warshall Case Study: Experiment Setup

- Comparing two Floyd–Warshall encodings: parallel vs sequential
- Both versions store the distance matrix in memory
- Sequential version uses one worker; parallel version uses 4 block workers
- Random directed graphs with up to 15 nodes

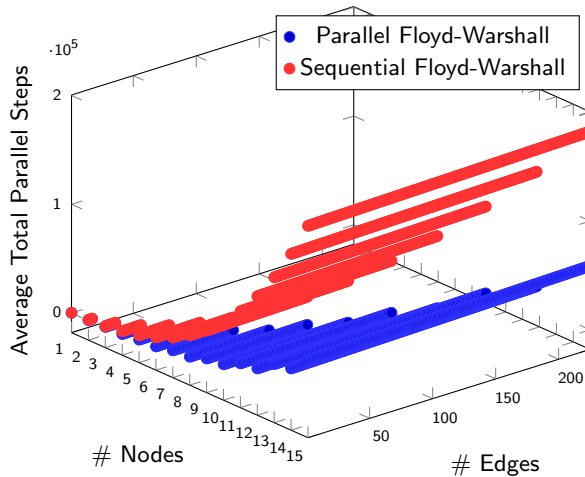
Floyd–Warshall Case Study: Experiment Setup

- Comparing two Floyd–Warshall encodings: parallel vs sequential
- Both versions store the distance matrix in memory
- Sequential version uses one worker; parallel version uses 4 block workers
- Random directed graphs with up to 15 nodes
- All edge counts from 1 to $n^2 - 1$ (where n is a number of nodes)

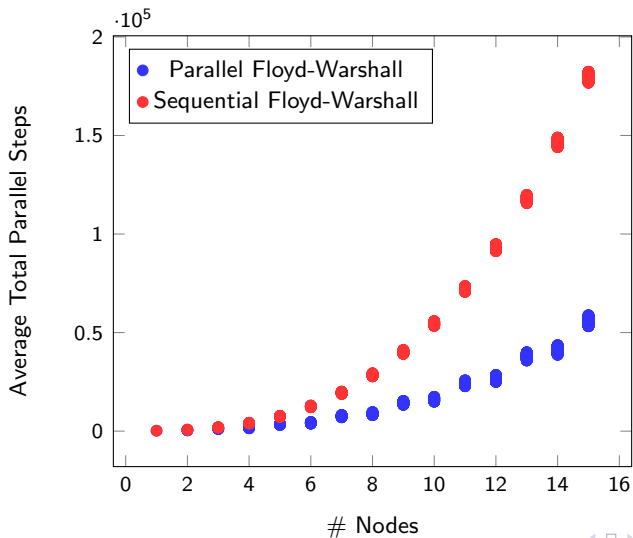
Floyd–Warshall Case Study: Experiment Setup

- Comparing two Floyd–Warshall encodings: parallel vs sequential
- Both versions store the distance matrix in memory
- Sequential version uses one worker; parallel version uses 4 block workers
- Random directed graphs with up to 15 nodes
- All edge counts from 1 to $n^2 - 1$ (where n is a number of nodes)
- Metric: total number of parallel reduction steps

Floyd–Warshall Case Study: Parallel Reduction Steps



Floyd–Warshall Case Study: Parallel Reduction Steps



Floyd–Warshall Case Study: Interpretation of Results

- Number of **vertices** has much stronger impact than number of edges

Floyd–Warshall Case Study: Interpretation of Results

- Number of **vertices** has much stronger impact than number of edges
- Edge count mainly affects graph-to-matrix **initialization**

Floyd–Warshall Case Study: Interpretation of Results

- Number of **vertices** has much stronger impact than number of edges
- Edge count mainly affects graph-to-matrix **initialization**
- Parallel version requires noticeably **fewer** parallel reduction steps

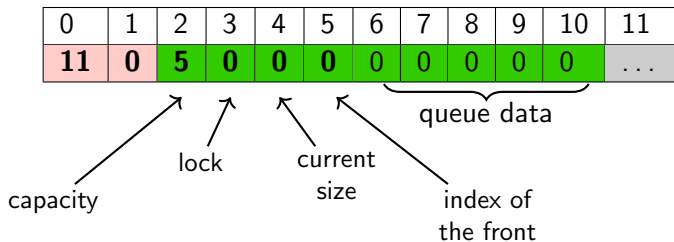
Floyd–Warshall Case Study: Interpretation of Results

- Number of **vertices** has much stronger impact than number of edges
- Edge count mainly affects graph-to-matrix **initialization**
- Parallel version requires noticeably **fewer** parallel reduction steps
- Reason: matrix entries are updated in parallel by **4 workers**

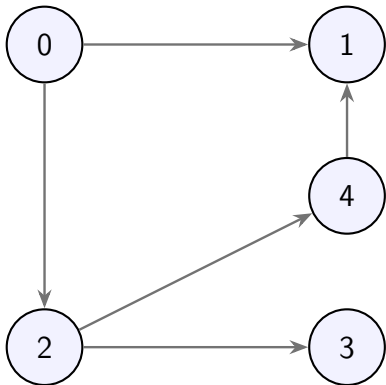
Floyd–Warshall Case Study: Interpretation of Results

- Number of **vertices** has much stronger impact than number of edges
- Edge count mainly affects graph-to-matrix **initialization**
- Parallel version requires noticeably **fewer** parallel reduction steps
- Reason: matrix entries are updated in parallel by **4 workers**
- Observed behavior **matches** the expected $\mathcal{O}(V^3/p)$ parallel complexity

Concurrent Queues in MemTRS: Using Synchronization Primitives



Parallel Breadth-First Search

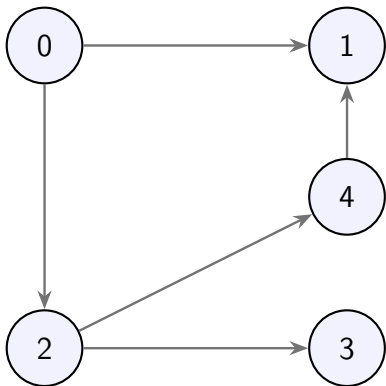


Address	Value
0	2
1	0

visited queue @ 8 queue @ 38 worker

Init heap metadata

Parallel Breadth-First Search

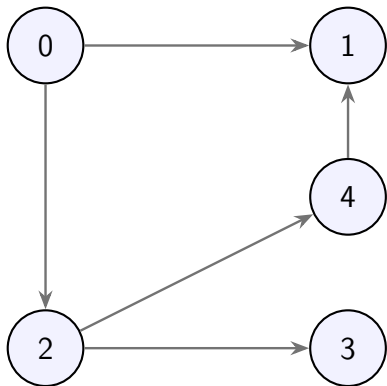


Address	Value
0	8
1	0
2	5
3-7	0, 0, 0, 0, 0

visited queue @ 8 queue @ 38 worker

Allocate visited array

Parallel Breadth-First Search

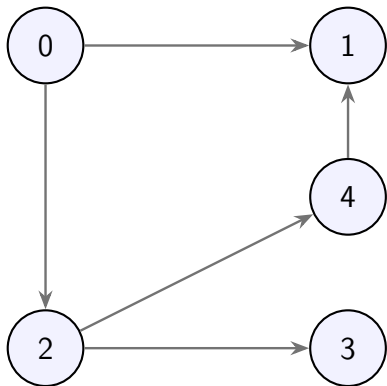


Address	Value
0	38
1	0
2	5
3-7	0, 0, 0, 0, 0
8-11	26, 0, 0, 0
12	0
13	0
14	0
15-37	0, ..., 0

visited queue @ 8 queue @ 38 worker

Allocate queue @ 8

Parallel Breadth-First Search

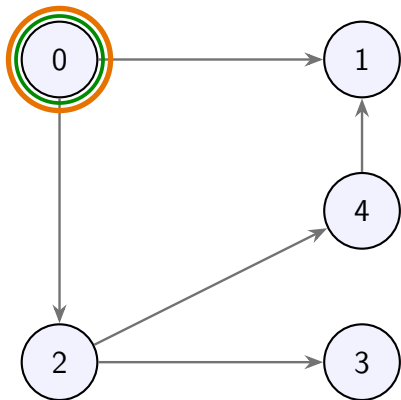


visited queue @ 8 queue @ 38 worker

Allocate queue @ 38

Address	Value
0	68
1	0
2	5
3-7	0, 0, 0, 0, 0
8-11	26, 0, 0, 0
12	0
13	0
14	0
15-37	0, ..., 0
38-41	26, 0, 0, 0
42	0
43	0
44-67	0, ..., 0

Parallel Breadth-First Search

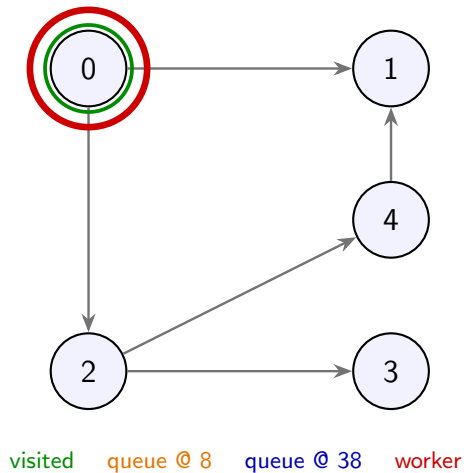


visited queue @ 8 queue @ 38 worker

Seed: visit 0, push 0 to Q8

Address	Value
0	68
1	0
2	5
3-7	1, 0, 0, 0, 0
8-11	26, 0, 1, 0
12	0
13	0
14	0
15-37	0, ..., 0
38-41	26, 0, 0, 0
42	0
43	0
44-67	0, ..., 0

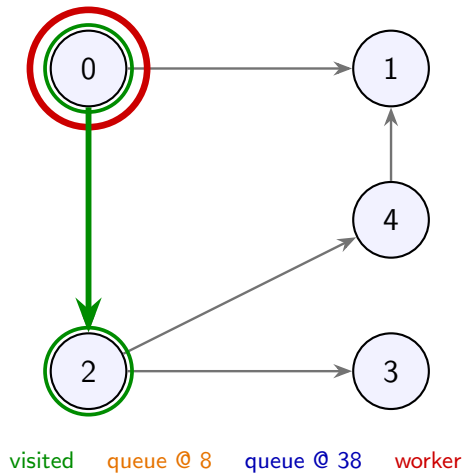
Parallel Breadth-First Search



Pop 0 from Q8

Address	Value
0	68
1	0
2	5
3-7	1, 0, 0, 0, 0
8-11	26, 0, 0, 1
12	0
13	0
14	0
15-37	0, ..., 0
38-41	26, 0, 0, 0
42	0
43	0
44-67	0, ..., 0

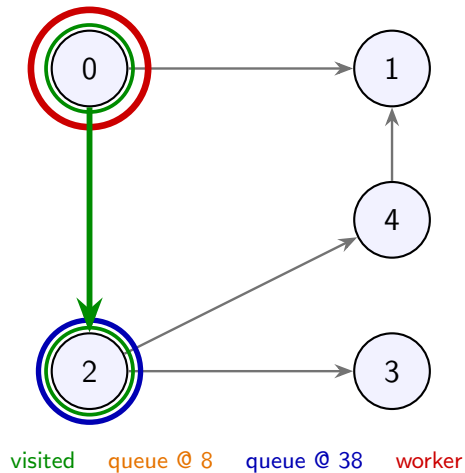
Parallel Breadth-First Search



Discover 2: set visited

Address	Value
0	68
1	0
2	5
3-7	1, 0, 1, 0, 0
8-11	26, 0, 0, 1
12	0
13	0
14	0
15-37	0, ..., 0
38-41	26, 0, 0, 0
42	0
43	0
44-67	0, ..., 0

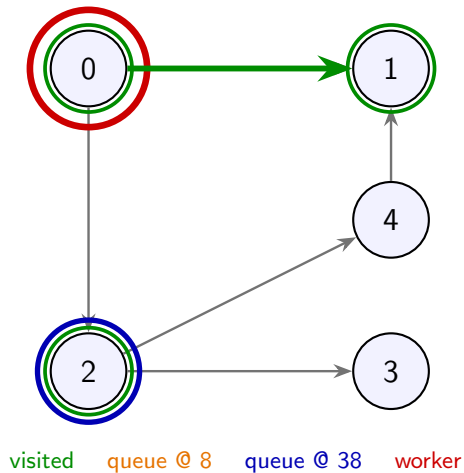
Parallel Breadth-First Search



Push 2 to Q38

Address	Value
0	68
1	0
2	5
3-7	1, 0, 1, 0, 0
8-11	26, 0, 0, 1
12	0
13	0
14	0
15-37	0, ..., 0
38-41	26, 0, 1, 0
42	2
43	0
44-67	0, ..., 0

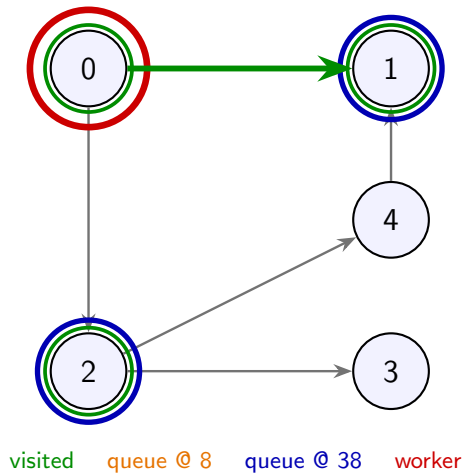
Parallel Breadth-First Search



Discover 1: set visited

Address	Value
0	68
1	0
2	5
3-7	1, 1, 1, 0, 0
8-11	26, 0, 0, 1
12	0
13	0
14	0
15-37	0, ..., 0
38-41	26, 0, 1, 0
42	2
43	0
44-67	0, ..., 0

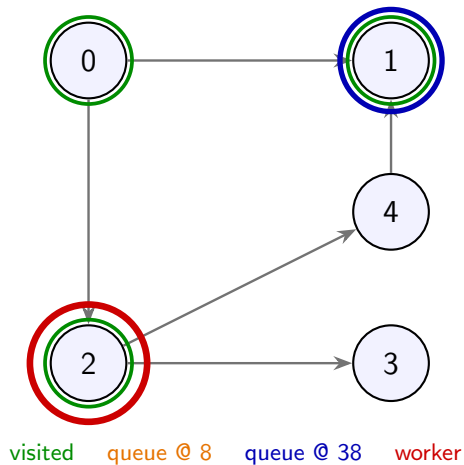
Parallel Breadth-First Search



Push 1 to Q38

Address	Value
0	68
1	0
2	5
3-7	1, 1, 1, 0, 0
8-11	26, 0, 0, 1
12	0
13	0
14	0
15-37	0, ..., 0
38-41	26, 0, 2, 0
42	2
43	1
44-67	0, ..., 0

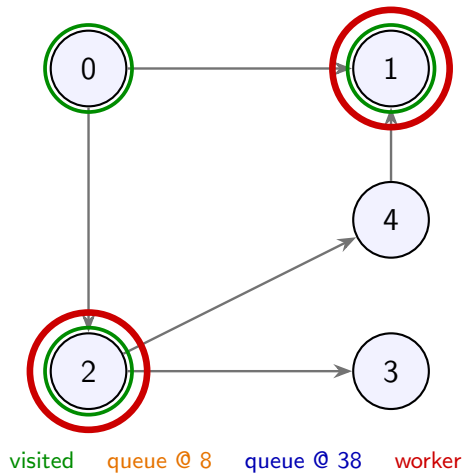
Parallel Breadth-First Search



Pop 2 from Q38

Address	Value
0	68
1	0
2	5
3-7	1, 1, 1, 0, 0
8-11	26, 0, 0, 1
12	0
13	0
14	0
15-37	0, ..., 0
38-41	26, 0, 1, 1
42	2
43	1
44-67	0, ..., 0

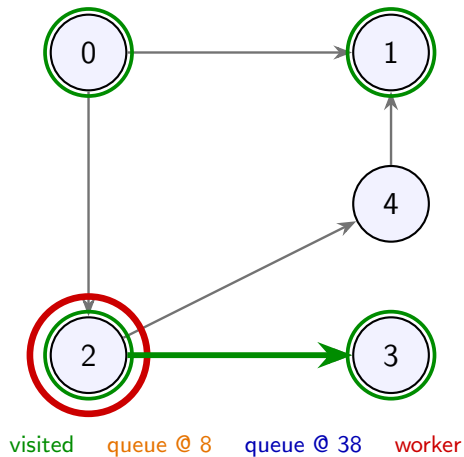
Parallel Breadth-First Search



Pop 1 from Q38

Address	Value
0	68
1	0
2	5
3-7	1, 1, 1, 0, 0
8-11	26, 0, 0, 1
12	0
13	0
14	0
15-37	0, ..., 0
38-41	26, 0, 0, 2
42	2
43	1
44-67	0, ..., 0

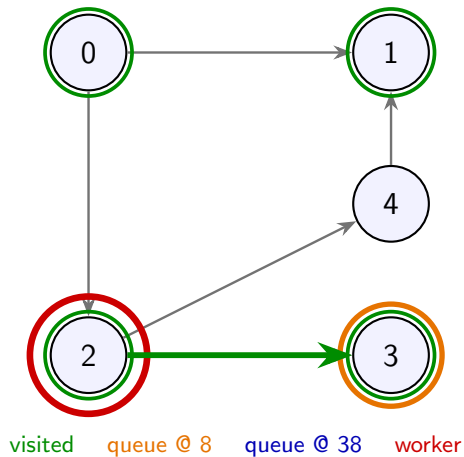
Parallel Breadth-First Search



Discover 3: set visited

Address	Value
0	68
1	0
2	5
3-7	1, 1, 1, 1, 0
8-11	26, 0, 0, 1
12	0
13	0
14	0
15-37	0, ..., 0
38-41	26, 0, 0, 2
42	2
43	1
44-67	0, ..., 0

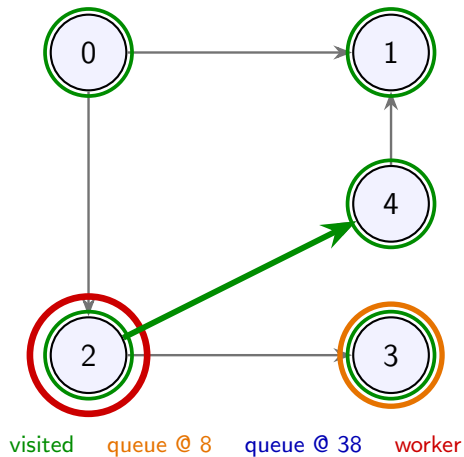
Parallel Breadth-First Search



Push 3 to Q8

Address	Value
0	68
1	0
2	5
3-7	1, 1, 1, 1, 0
8-11	26, 0, 1, 1
12	0
13	3
14	0
15-37	0, ..., 0
38-41	26, 0, 0, 2
42	2
43	1
44-67	0, ..., 0

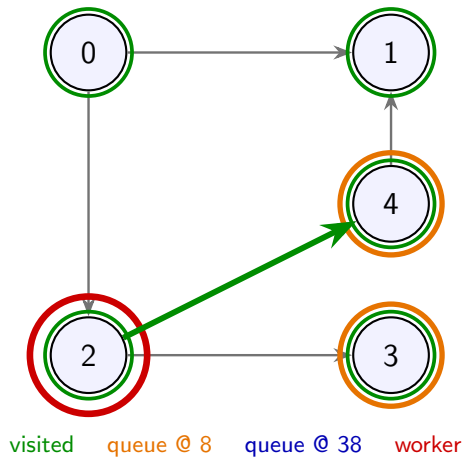
Parallel Breadth-First Search



Discover 4: set visited

Address	Value
0	68
1	0
2	5
3-7	1, 1, 1, 1, 1
8-11	26, 0, 1, 1
12	0
13	3
14	0
15-37	0, ..., 0
38-41	26, 0, 0, 2
42	2
43	1
44-67	0, ..., 0

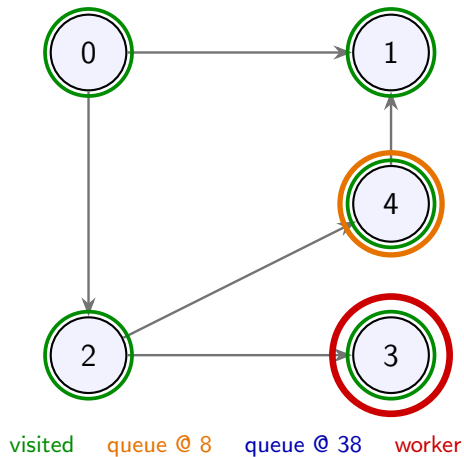
Parallel Breadth-First Search



Push 4 to Q8

Address	Value
0	68
1	0
2	5
3-7	1, 1, 1, 1, 1
8-11	26, 0, 2, 1
12	0
13	3
14	4
15-37	0, ..., 0
38-41	26, 0, 0, 2
42	2
43	1
44-67	0, ..., 0

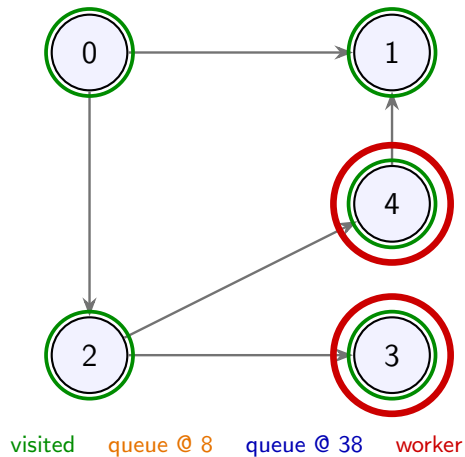
Parallel Breadth-First Search



Pop 3 from Q8

Address	Value
0	68
1	0
2	5
3-7	1, 1, 1, 1, 1
8-11	26, 0, 1, 2
12	0
13	3
14	4
15-37	0, ..., 0
38-41	26, 0, 0, 2
42	2
43	1
44-67	0, ..., 0

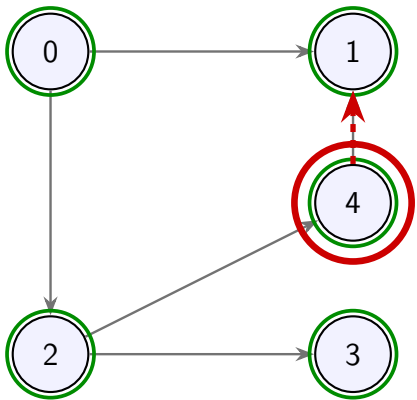
Parallel Breadth-First Search



Pop 4 from Q8

Address	Value
0	68
1	0
2	5
3-7	1, 1, 1, 1, 1
8-11	26, 0, 0, 3
12	0
13	3
14	4
15-37	0, ..., 0
38-41	26, 0, 0, 2
42	2
43	1
44-67	0, ..., 0

Parallel Breadth-First Search

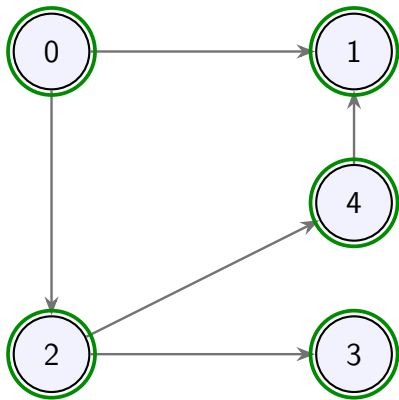


visited queue @ 8 queue @ 38 worker

4 → 1 fails: 1 already visited

Address	Value
0	68
1	0
2	5
3-7	1, 1, 1, 1, 1
8-11	26, 0, 0, 3
12	0
13	3
14	4
15-37	0, ..., 0
38-41	26, 0, 0, 2
42	2
43	1
44-67	0, ..., 0

Parallel Breadth-First Search



visited queue @ 8 queue @ 38 worker

Done!

Address	Value
0	68
1	0
2	5
3-7	1, 1, 1, 1, 1
8-11	26, 0, 0, 3
12	0
13	3
14	4
15-37	0, ..., 0
38-41	26, 0, 0, 2
42	2
43	1
44-67	0, ..., 0

Sequential Breadth-First Search: Comparison Baseline

- Graph as nested linked lists

```
consG(cons(1, cons(2, nil)),  
      consG(nil,  
            consG(cons(3, cons(4, nil)),  
                  consG(nil,  
                        consG(cons(1, nil),  
                              nilG))))))
```

Sequential Breadth-First Search: Comparison Baseline

- Graph as nested linked lists
- Visited array as a linked list

```
cons(1, cons(0, cons(1, cons(0, cons(0, nil)))))
```

Sequential Breadth-First Search: Comparison Baseline

- Graph as nested linked lists
- Visited array as a linked list
- Queue as a linked-list term

cons(2, **cons**(2, **cons**(1, nil)))

Sequential Breadth-First Search: Comparison Baseline

- Graph as nested linked lists
- Visited array as a linked list
- Queue as a linked-list term
- Linear **pushBack** traversal

pushBack(e , **cons**(u , l)) $\rightarrow$ **cons**(u , **pushBack**(e , l))

pushBack(e , nil) $\rightarrow$ **cons**(e , nil)

Sequential Breadth-First Search: Comparison Baseline

- Graph as nested linked lists
- Visited array as a linked list
- Queue as a linked-list term
- Linear **pushBack** traversal
- Sequential processing

Baseline idea

Graphs, visited states, and queues all live inside terms. The cost of updates is paid by rebuilding term structure.

baseline = large terms + sequential processing

Breadth-First Search Case Study: Experiment Setup

- Comparing two BFS encodings: parallel MemTRS vs sequential list-based baseline

Breadth-First Search Case Study: Experiment Setup

- Comparing two BFS encodings: parallel MemTRS vs sequential list-based baseline
- Randomly generated directed graphs

Breadth-First Search Case Study: Experiment Setup

- Comparing two BFS encodings: parallel MemTRS vs sequential list-based baseline
- Randomly generated directed graphs
- Number of nodes: $n = 1, 2, \dots, 15$
- Number of edges: $1, 2, \dots, n^2 - 1$

Breadth-First Search Case Study: Experiment Setup

- Comparing two BFS encodings: parallel MemTRS vs sequential list-based baseline
- Randomly generated directed graphs
- Number of nodes: $n = 1, 2, \dots, 15$
- Number of edges: $1, 2, \dots, n^2 - 1$
- Starting vertex: 0

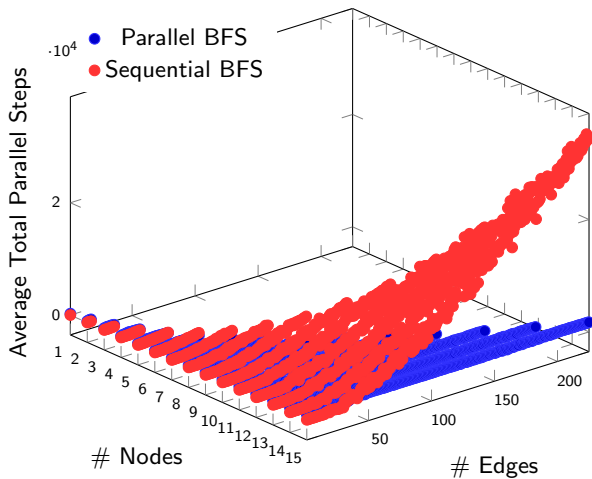
Breadth-First Search Case Study: Experiment Setup

- Comparing two BFS encodings: parallel MemTRS vs sequential list-based baseline
- Randomly generated directed graphs
- Number of nodes: $n = 1, 2, \dots, 15$
- Number of edges: $1, 2, \dots, n^2 - 1$
- Starting vertex: 0
- Repeated 3 times for each configuration

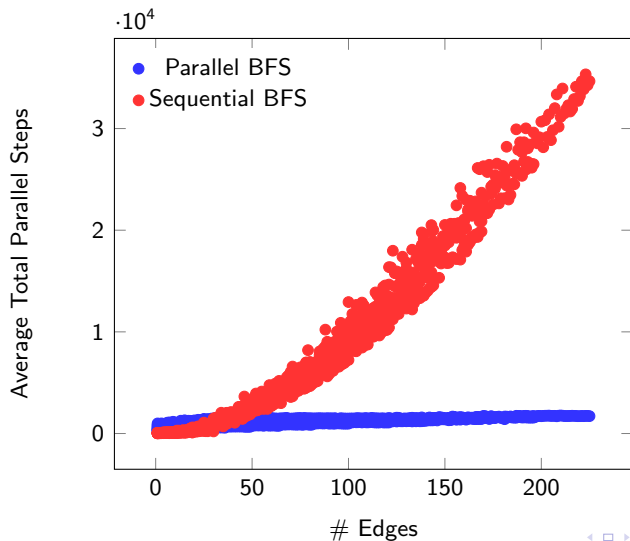
Breadth-First Search Case Study: Experiment Setup

- Comparing two BFS encodings: parallel MemTRS vs sequential list-based baseline
- Randomly generated directed graphs
- Number of nodes: $n = 1, 2, \dots, 15$
- Number of edges: $1, 2, \dots, n^2 - 1$
- Starting vertex: 0
- Repeated 3 times for each configuration
- Main metric: total number of parallel reduction steps

Breadth-First Search Case Study: Parallel Reduction Steps



Breadth-First Search Case Study: Parallel Reduction Steps



Breadth-First Search Case Study: Interpretation of Results

- Parallel MemTRS BFS requires far **fewer** parallel reduction steps

Breadth-First Search Case Study: Interpretation of Results

- Parallel MemTRS BFS requires far **fewer** parallel reduction steps
- For very sparse graphs, parallel BFS has **scheduling overhead** from **bfsWorkers**

Breadth-First Search Case Study: Interpretation of Results

- Parallel MemTRS BFS requires far **fewer** parallel reduction steps
- For very sparse graphs, parallel BFS has **scheduling overhead** from **bfsWorkers**
- Sequential baseline grows **faster** as graphs become denser

Breadth-First Search Case Study: Interpretation of Results

- Parallel MemTRS BFS requires far **fewer** parallel reduction steps
- For very sparse graphs, parallel BFS has **scheduling overhead** from **bfsWorkers**
- Sequential baseline grows **faster** as graphs become denser
- Main reason: visited-state lookup/update and **pushBack** are **linear** over linked lists

Breadth-First Search Case Study: Interpretation of Results

- Parallel MemTRS BFS requires far **fewer** parallel reduction steps
- For very sparse graphs, parallel BFS has **scheduling overhead** from **bfsWorkers**
- Sequential baseline grows **faster** as graphs become denser
- Main reason: visited-state lookup/update and **pushBack** are **linear** over linked lists
- Parallel BFS scales **better** with the number of edges

Reflection: What Is Still Difficult?

- MemTRS can model **realistic** shared-memory algorithms

Reflection: What Is Still Difficult?

- MemTRS can model **realistic** shared-memory algorithms
- But the user must still reason **manually** about races, evaluation order, and **CAS**-synchronization

Reflection: What Is Still Difficult?

- MemTRS can model **realistic** shared-memory algorithms
- But the user must still reason **manually** about races, evaluation order, and **CAS**-synchronization
- Termination analysis is **not** a direct extension of LCTRS

Reflection: What Is Still Difficult?

- MemTRS can model **realistic** shared-memory algorithms
- But the user must still reason **manually** about races, evaluation order, and **CAS**-synchronization
- Termination analysis is **not** a direct extension of LCTRS
- Scheduling, locks, and busy waiting **complicate** the meaning of termination

Reflection: What Is Still Difficult?

- MemTRS can model **realistic** shared-memory algorithms
- But the user must still reason **manually** about races, evaluation order, and **CAS**-synchronization
- Termination analysis is **not** a direct extension of LCTRS
- Scheduling, locks, and busy waiting **complicate** the meaning of termination
- Future work should focus on abstractions and analysis techniques

Research Project Summary

Main Research Question Revisited

Can we extend term rewriting with explicit shared memory so that it is **formal**, **implementable**, and **useful for modelling concurrent algorithms**?

- ✓ Introduced **Memory Term Rewriting Systems (MemTRS)**

Research Project Summary

Main Research Question Revisited

Can we extend term rewriting with explicit shared memory so that it is **formal**, **implementable**, and **useful for modelling concurrent algorithms**?

- ✓ Introduced **Memory Term Rewriting Systems (MemTRS)**
- ✓ Formalized memory operations, side effects, and parallel reduction

Research Project Summary

Main Research Question Revisited

Can we extend term rewriting with explicit shared memory so that it is **formal**, **implementable**, and **useful for modelling concurrent algorithms**?

- ✓ Introduced **Memory Term Rewriting Systems (MemTRS)**
- ✓ Formalized memory operations, side effects, and parallel reduction
- ✓ Analysed nondeterminism, data races, and CAS-based synchronization

Research Project Summary

Main Research Question Revisited

Can we extend term rewriting with explicit shared memory so that it is **formal**, **implementable**, and **useful for modelling concurrent algorithms**?

- ✓ Introduced **Memory Term Rewriting Systems (MemTRS)**
- ✓ Formalized memory operations, side effects, and parallel reduction
- ✓ Analysed nondeterminism, data races, and CAS-based synchronization
- ✓ Modelled arrays, matrices, concurrent queues, sorting, BFS, and Floyd–Warshall

Research Project Summary

Main Research Question Revisited

Can we extend term rewriting with explicit shared memory so that it is **formal**, **implementable**, and **useful for modelling concurrent algorithms**?

- ✓ Introduced **Memory Term Rewriting Systems (MemTRS)**
- ✓ Formalized memory operations, side effects, and parallel reduction
- ✓ Analysed nondeterminism, data races, and CAS-based synchronization
- ✓ Modelled arrays, matrices, concurrent queues, sorting, BFS, and Floyd–Warshall
- ✓ Implemented MemTRS in *Cora* and evaluated it through experiments

Thank you!

Do you have any questions?